

CUDA Kernel Library: Debug Report

Olajide Badejo

2026

Abstract

This is the companion to the main report: the dated record of the real problems hit while building the library, grouped by theme. Each entry is symptom, root cause, the fix and why, and how it was verified. The failed diagnostic round, where a plausible change measured worse and was reverted, is here too as an equal rank result. This document is generated from the same engineering log the work was recorded in as it happened.

1 Toolchain and build

CMake targeted the wrong architecture. The first device probe build compiled sm_75 despite the top CMakeLists setting the architecture to 120. Root cause: the architecture was set after the project command, inside a guard that was already false because project seeds a default. Fix: set it before project so enable_language reads it. Verified by the rebuild compiling sm_120 and running on the card.

CUDA 13 removed clockRate and memoryClockRate. The probe failed to compile because both fields were removed from cudaDeviceProp in CUDA 13 (deprecated in 12.x). Fix: query the same values through cudaDeviceGetAttribute. Verified: the probe prints 14001 MHz memory clock and 2625 MHz core clock, matching the datasheet.

Pedantic warnings on generated stubs. With the pedantic warning set forwarded to the host compiler, the build failed on line directives inside nvcc generated stub files under separable compilation. Fix: keep the pedantic flag on hand written host code only, and forward just the standard warning set to the nvcc host pass. Device code still gets warnings as errors. Verified by a clean build.

Deprecated NVML calls. The telemetry monitor drew deprecation warnings from the CUDA 13 NVML for the temperature and throttle reason queries. Fix: move to the versioned temperature struct and the events reason query. Verified: warning free build, the zero warning gate holds.

2 The ncu permission fight in WSL2

ERR_NVGPUCTRPERM, and root did not help. Nsight Compute refused to read GPU performance counters for both the normal user and root. Root cause: under WSL2 the CUDA driver is the Windows driver reached through the paravirtualization layer, so counter access is gated by the Windows driver policy, not by any Linux user. The diagnostic rounds are the core of the project and depend on this, so it had to be cleared first. Fix: on the Windows side, set the driver policy to allow all users to read the counters (the registry value under the nvlddmkm key), then reboot. Verified: the ncu probe on a trivial copy kernel then returned no error and a populated table (DRAM throughput 86 percent, the expected memory bound signature of a copy), and the first diagnostic round ran cleanly.

3 A diagnostic round that failed

The three stage pipeline made the top kernel slower. Hypothesis: the top tensor kernel's residual latency exposure came from too shallow a software pipeline, so deepen the cp.async pipeline from two stages to three. Result: slower, from about 80 to 74 percent of cuBLAS. Root cause: the third shared buffer raised shared memory use and cut occupancy, and the two stage pipeline was already hiding the global load latency, so the depth was spent on the wrong bottleneck. Fix: reverted to the two stage kernel. The genuine lever, found in the next round, was a shared memory swizzle to remove bank conflicts. This entry is kept because the reverted change is as much a result as the one that worked: it is the evidence that the swizzle, not more pipelining, was the right call.